Near-optimal space in GreenBubble – a quick update

What I actually copied from your repo, how the model works, and where the comparison landed

Alberto Alamia

August 2026

1 What I actually copied from your repo – direct mapping

To be really explicit (this is the bit I want your sign-off on before the branch goes public): most of the "near-optimal" code in GB I use is PyPSA's *own* public MGA API (Fabian Neumann's implementation of Neumann & Brown 2021, inside PyPSA) – that's not your code at all, just PyPSA's. Separately from that, on my newest branch (`near-optimal_dev3`) I did go and literally vendor pieces of *your* repo (`github.com/aleks-g/intersecting-near-opt-spaces`, tag `v1.0.1`, GPL-3.0-or-later). Table below is the , function-by-function account. I used Claude for interpreting your code, and figure out how to adapt it to my model.

Your repo (file / function)	Where it landed in GreenBubble	What I actually did
`geometry.py`: `ch_centre()`, `intersection()`, `facet_normals()`, hypersphere samplers	`scripts/vendor/near_opt_geometry.py`	**Vendored verbatim** (same structure, docstrings, algorithm), your licence & attribution kept, in a module kept separate from my MIT code. Only change: your Gurobi LP calls → `scipy.optimize.linprog`.
`compute_near_opt.py`: the adaptive facet / touching-ball direction sampler (volume- and centre-shift convergence)	same file, `explore_hull_adaptive()`	**Vendored verbatim**, same convergence criteria (`conv_eps`/`conv_iter`); only the per-direction *solve* underneath is mine (see row below), since your code solves via the old `pypsa.linopf` API in parallel workers.
`utilities.py` / `solve_operations.py` / `summarise_feasibility.py`: `set_extendable_false`, `compute_feasibility_criteria`	`scripts/vendor/near_opt_feasibility.py`	**Vendored verbatim**, same GPL treatment, used for the simultaneous multi-year feasibility check (Tier 3 validation, Section 4).
`utilities.py`: `solve_network_in_direction`	`scripts/near_optimal.py`: `_solve_mga_in_direction`	**Same algorithm, re-typed**: yours targets PyPSA's old `linopf`/`linopt` API, which doesn't run on the PyPSA version I'm on (1.0.7, Linopy-based). I had to re-derive the same steps against `n.optimize`, mainly so I could re-inject GreenBubble's own custom constraints (Section 2.1) into the middle of it – your version has no hook for that either, so this was never really optional.
`calc_obj_bound.py`: shared bound $c^\star = (1+\varepsilon)\max_i c_{\text{opt}}(i)$	`near_optimal.py`: `_add_budget_constraint`	**Re-derived from the paper's eq. 9, not copied** it was confirmed identical once it was compared. Generalised with a $\lvert\cdot\rvert$ so it also works for my price-driven, sometimes-negative objective (Section 2) – the origianl code has no analogue for that, since your objective is always a positive investment cost.
`compute_robust_conservative/mean/naive.py`	—	**Not ported.** I only implemented your "exact" realisation strategy.
`prepare_validation.py` (cardinal-perturbation robustness study)	—	**Not ported** – noted as available, not requested for this comparison.

Everything else (dimension auto-derivation from GreenBubble's own tech config, the Snakemake staging, plotting, the sign-robust budget, the whole demand/price-mode handling below) is GreenBubble-original and has no counterpart in your repo.

The ask: before I push `near-optimal_dev3` and open the GreenBubble repo up, could you have a quick look at `scripts/vendor/near_opt_geometry.py` and `scripts/vendor/near_opt_feasibility.py` and confirm you're OK with this scope of reuse (vendored under GPL, kept in its own module, your licence header kept)? Want to make sure I'm not overstepping before it's public.

2 How GreenBubble actually works (the bit that matters for reading the results)

GreenBubble models an industrial Power-to-X hub (electrolysis, methanol synthesis, biogas upgrading/methanation, storage) as a PyPSA network with internal buses per energy carrier (electricity, H_2, CO_2, bioCH_4, methanol, district heat, low/mid-temp heat).

It can run in **two modes**, and this matters a lot for how to read an objective value:

- **demand-driven** – fixed product demands (H_2, CH_4, methanol) have to be met, objective = cost to be minimised (but side sales could push it to negative as revenue). **This is the mode used for everything in this note.**
- **price-driven** – no fixed demand, products are sold at an exogenous price and the model maximises profit, which in the LP shows up as *minimising* a cost objective that's typically *negative* (revenue > cost).

On top of the main PtX products, GreenBubble also lets a few other internal streams be sold externally (side sales) at a market price the same way: electricity (always active, both buy and sell), district heating (link exists in the model, **not activated** in these runs), and captured CO_2 for sequestration (also modelled, **not activated** here). Only electricity export was live for anything discussed below.

2.1 Why I needed the `max_RE_to_grid` constraint

Because electricity export is priced at the real, exogenous spot price, a demand-driven solve has a dangerous escape: it could freely expand renewable capacity *beyond* what the PtX processes actually need, purely to sell the surplus into the grid whenever the spot price is good. Nothing in the demand constraints stops that, and it can make the capacity-expansion problem numerically fragile or push it toward a design that's really an arbitrage play rather than a PtX plant. So I added a constraint that ties allowed RE sales to actual on-site consumption:

$$\sum_t \Delta t \; p_t^{\text{export}} \quad \leq \quad \alpha \cdot \sum_t \Delta t \; p_t^{\text{consumed}} \tag{1}$$

i.e. total electricity exported to the grid over the year is capped at a fraction α (`max_RE_to_grid`) of total electricity drawn on-site by the PtX agents (electrolysis, methanolisation, etc. – the export link itself is excluded from the right-hand side). Default is $\alpha = 0.30$ (30%); this constraint is applied as a custom Linopy constraint (`add_max_RE_sales_constraint` in `helpers.py`) added on top of every solve, and – this is the whole engineering headache in Section 3 below – it has to be re-applied inside every single MGA direction solve too, or the near-optimal space gets computed against the wrong (looser) feasible region.

3 Three tiers implementaiton of the NOS analysis in GreenBubble

Tier 1 – ranges. For each selected technology (installed capacity, MW), minimise and maximise it alone subject to staying within slack ε of the cost optimum. Tells you must-have (min > 0) vs. must-avoid (max $\approx$ 0) technologies. *Mine*: everything, PyPSA's own API mechanics underneath.

Tier 2 – hull. Sample directions in the reduced capacity space, solve each, take the convex hull of the extreme points – the approximated near-optimal polytope. *On* `dev3`: the direction sampler is **your** adaptive facet/Chebyshev-ball code (Table 1), not my earlier fixed-count Halton sampling – and it matters: same problem, **3.6× larger** Chebyshev radius found for a comparable solve budget (0.58 → 2.08 on my 3-year test case).

Tier 3 – robustness. Solve the optimum and hull independently for several weather/price years, intersect the per-year polytopes, take the **Chebyshev centre** of the intersection as the design that's simultaneously near-optimal across all years. *Geometry (centre, intersection) is your code, vendored*; the shared bound $c^\star = \max_i c_{\text{opt}}(i)$ is my re-derivation of your eq. 9; and the simultaneous multi-year feasibility check that follows (fix the realised design non-extendable, re-dispatch against each year's real network, load-shedding safety valve) ports your `solve_operations.py`/`summarise_feasibility.py` logic.

I also mirrored your rule decomposition (`compute_optimum` → `mga` → `compute_near_opt` → `intersect_near_optimal` → `compute_robust_*`) as a separate staged Snakemake pipeline, so re-running e.g. the intersection after a tuning change doesn't force re-solving three years of networks from scratch – same caching benefit your split gives you.

4 The comparison and a issue I didn't expect

For the NOS intersect run I build one capacity mix that's good across **seven** weather/price years (2019–2025), demand-driven, comparing your Chebyshev-centre robust design against an equal-weight 7-scenario PyPSA stochastic solve of the same problem. Dimensions: `[onwind, solar, AEC electrolysis, methanolisation, battery]`.

The issue: my first attempts came back with an **empty** 7-year intersection – no design at all satisfied ε-near-optimality in all seven years simultaneously, at either 5% or 7% slack. Digging in: 2022's cost-optimal objective was roughly *half* every other year's (28 M€/y vs. 53–61 M€/y elsewhere), and its optimal design sat in a completely different part of capacity space (159.6 MW solar / 125.1 MW battery vs. ≤7 MW in every other year). The reason: 2022 is the European energy-price-crisis year, with extreme spot-price spikes, and at the default 30% export cap the model could earn almost 56 M€/y purely from RE-export arbitrage – nearly twice 2022's entire net objective. That single year's objective got pulled so far away from the other six by price-arbitrage revenue that its near-optimal region stopped overlapping with theirs at all.

I fixed this pragmatically by **tightening** α (`max_RE_to_grid`) until 2022 stopped being such an outlier:

α	2022 objective [M€/y]	Objective spread across 7y [M€/y]	2022 solar/battery [MW]
0.30 (default)	28.09 (outlier)	33.0	159.6 / 125.1
0.10	50.95 (still lowest)	12.7	62.7 / 42.4
0.05	58.19 (mid-pack)	12.2	43.5 / 26.5

At $\alpha = 0.05$, 2022 was no longer the odd one out, so I committed to the full pipeline at $\alpha = \varepsilon = 0.05$ and got a real, **feasible** intersection this time.

Is this a real methodological gap, or just a GreenBubble quirk? I don't think it's fully GreenBubble-specific, and it's the thing I most want your take on. My understanding is that your method is based on the objecitve values for all the different years to be relatively close to each other and differ by margin similar to the slack. Otherwise the intersect will not really work. Am I correct? In your original setting, year-to-year variation is basically weather

(renewable resource availability) feeding into a *cost* objective that's always positive – a bad weather year is expensive, a good one is cheap, but nothing about it is structurally different across years. In GreenBubble, year-to-year variation carries *both* weather *and* market-price variation, and because the model is price-driven at the margins (grid sales at real spot prices), an unusual *price* year can dominate a year's whole objective in a way that has nothing to do with the underlying technology-robustness question Tier 3 is trying to answer – it's an economic outlier, not a physical one. Tightening α worked here, but it feels like patching the symptom (capping how much arbitrage revenue is even available) rather than something the NOS method itself accounts for. Curious whether this has come up for you, or whether there's a more principled way to handle a year whose objective is an outlier for price reasons rather than resource-availability reasons.

It is worth reminding that two-stages stochastic run over the 7 years (equiprobable) optimizes all the capacities not only the 5 used for the expansion and intersect of the NOS.

4.1 Trials, end to end

Run	α	ε	Ramp limits	**Outcome**
`...D`	0.30	0.05	active	Empty intersection (r = NaN)
`...D2`	0.30	0.07	nulled	Still empty – ramp limits weren't the driver
α-sweep diagnostics (cost-optimum only, no hull)	0.30/0.10/0.05	–	nulled	2022 converges back toward the group as $\alpha \downarrow$ (table above)
`...D4_a05s05`	0.05	0.05	nulled	**Feasible.** r = 1.1334; design onwind 137.2 / solar 29.5 / AEC 70.7 / meoh 3.1 / battery 26.5 MW; cost **65.39 M€/y**. Validated: ∼0.015% curtailment in *every* one of the 7 years incl. 2022
`...S2` (stochastic, matched α = 0.05, EVPI on)	0.05	– (no slack concept)	nulled	RP = 63.978 M€/y, E[WS] = 60.971, **EVPI = 3.007 M€/y** (≈4.7% of RP)

Bottom line: robust (Chebyshev-centre) design costs 65.39 M€/y, the stochastic expected-cost optimum costs 63.98 M€/y – a **1.41 M€/y (2.2%)** premium for a design that's, simultaneously good in all seven years (near-zero curtailment everywhere, including 2022), instead of just cheap on average with no guarantee for any single year. Small premium for a real guarantee, which is a nice result – but only once α was tuned away from the arbitrage issue above, which is the part I'd like to sanity-check with you.

4.2 Coordinates side by side

Worth looking at the actual capacities behind those two numbers, not just the objective values – perhaps it is not a general case but there seems to be a pattenr driving some key difference among the coordinates of the two approaches:

Dimension	Chebyshev centre (NOS)	Stochastic (RP)	Difference
onwind	137.18 MW	146.99 MW	−9.81 MW
solar	29.47 MW	0.50 MW	+28.97 MW
AEC	70.73 MW	85.09 MW	−14.36 MW
methanolisation	3.12 MW	1.95 MW	+1.17 MW
battery	26.52 MW	0.00 MW	+26.52 MW
Cost	65.39 M€/y	63.98 M€/y	+1.41 M€/y (2.2 %)

The stochastic optimum bets almost entirely on onwind+AEC and builds essentially **zero battery and negligible solar** – cheapest on average, since most of the seven years don't reward carrying solar/battery. The Chebyshev centre instead carries real solar and battery capacity specifically to stay inside 2022's own near-optimal band too (even at $\alpha = 0.05$, 2022's own unconstrained optimum still wants ∼43.5/26.5 MW solar/battery) – trading some onwind/AEC for that hedge, at the 1.41 M€/y premium above.

These are the coordinates for every single solution of the model in every energy year:

Year	onwind [MW]	solar [MW]	AEC [MW]	methanolisation [MW]	battery [MW]	Objective [M€/y]
2019	0.00	0.00	58.25	1.95	0.00	60.19
2020	0.00	0.00	64.66	1.95	0.00	53.05
2021	151.03	9.63	83.07	1.95	0.00	65.27
2022	159.77	43.47	83.68	2.03	26.52	58.19
2023	149.07	0.67	91.47	1.95	0.00	64.68
2024	125.19	0.00	84.43	1.95	0.00	63.81
2025	128.44	0.00	79.86	1.95	0.00	61.60

These are each year's own unconstrained cost-optimal design (the per-year optima behind the seven Tier-3 hulls), not the single realised Chebyshev centre above. Reading down the solar/battery columns explains where the Chebyshev centre's hedge comes from: essentially only 2021 and 2022 want any solar, and only 2022 wants any battery at all – every other year is happy with (near-)zero of both. The average of this table's seven objectives is 60.971 M€/y, which is exactly E[WS] from the EVPI calculation above (Section 4) – I checked directly (same objective value and, component by component, the identical solved capacities) and it's not a coincidence: `nos_year_optimum` and the stochastic side's per-year wait-and-see solve are the same deterministic per-year cost-optimal model, just invoked from two different code paths, so they have to agree.

4.3 Grid connection: import vs. export capacity (a result, not a dimension)

Not one of the five MGA dimensions, but a natural thing to read off both solved networks – the external grid connection's sized capacity, import (`DK1_to_El_*`, one link per on-site agent, summed) and export (`El3_to_DK1`, priced at real spot prices):

Grid connection	Chebyshev centre (NOS)	Stochastic (RP)	Difference
Import (purchase)	44.12 MW	40.07 MW	+4.04 MW
Export (sale)	19.08 MW	27.87 MW	−8.79 MW

The stochastic design leans harder on export (27.9 vs. 19.1 MW) despite having no battery to smooth output – more exposed to raw onwind variability, so it simply sells the surplus. The Chebyshev centre's battery lets it shift surplus in time instead, needing less export capacity but slightly more import capacity to keep the electrolyser firm through periods its own solar+battery pairing doesn't cover. `n_config`'s `grid connection: max capacity` is `.inf` (unset, base default) in both runs, so neither of these figures is bumping against an artificial ceiling – the sizing is purely economic.

And per year, behind those two aggregate designs – the same seven per-year cost-optimal networks as the coordinates table above, alongside each year's mean day-ahead spot price (the actual DK1 market series that both links' `marginal_cost` is built from, not a solved-network output):

Year	Import [MW]	Export [MW]	Mean spot price [€/MWh]
2019	59.75	0.00	38.49
2020	69.74	0.00	25.03
2021	35.08	30.66	88.14
2022	8.84	47.38	219.04
2023	41.54	20.27	86.83
2024	59.88	16.47	70.70
2025	69.10	30.26	81.10

2022's mean spot price (219 €/MWh) is roughly 2.5–9× every other year's – exactly the number behind the earlier arbitrage story. It is by far the cheapest year to skip importing (8.8 MW, barely bothering to buy from the grid) and by far the year building the most export capacity (47.4 MW) to capture that price level. 2021, the next most expensive year at 88 €/MWh, is the only other year that builds meaningful export capacity too (30.7 MW) – the same pattern showing up again, more mildly.

Alberto